Assignment: NumPy and Pandas

Section A: NumPy Array Manipulations

Question 1: The Outlier Cleaner

Function Name: `clean_outliers(array: np.ndarray) -> np.ndarray`

Description:
You are given a 1D NumPy array of integers or floats. Replace all outliers (values that are more than 2 standard deviations away from the mean) with the median of the array. Return the cleaned NumPy array after replacement.

Input:

- `array`: a 1D NumPy array of integers or floats

Output:

- A NumPy array representing the cleaned version of the input

Example:

```
Input:
array = np.array([50, 60, 250, 55, 70, 5])

Output:
array([50, 60, 70, 55, 70, 55])
```

.

Question 2: The Image Normalizer

Function Name: `normalize_image(image: np.ndarray) -> float`

Description:
Given a 16×16 NumPy array of random integers between 0 and 255, normalize each non-overlapping 4×4 block by subtracting its block mean. After normalization, compute and return the **overall standard deviation of the entire normalized array**.

Clarification:
Each 4×4 block is treated independently. Within each block:

$$\text{normalized\_block} = \text{block} - \text{mean(block)}$$

After performing this operation for all blocks, you have one combined 16×16 array of normalized values. Then compute the standard deviation of all 256 normalized values together.

Worked Example:

$$\text{Block} = \begin{bmatrix} 10 & 20 & 30 & 40 \\ 15 & 25 & 35 & 45 \\ 20 & 30 & 40 & 50 \\ 25 & 35 & 45 & 55 \end{bmatrix} \Rightarrow \text{Mean} = 32.5$$

Subtract the mean:

$$\begin{bmatrix} -22.5 & -12.5 & -2.5 & 7.5 \\ -17.5 & -7.5 & 2.5 & 12.5 \\ -12.5 & -2.5 & 7.5 & 17.5 \\ -7.5 & 2.5 & 12.5 & 22.5 \end{bmatrix}$$

Then compute the standard deviation across the full 16×16 normalized image:

$$\text{SD} = \sqrt{\frac{1}{256} \sum_{i=1}^{256} x_i^2}$$

Input:

- `image`: 2D NumPy array of shape (16, 16)

Output:

- A single float — the standard deviation of the final normalized image

Question 3: The Pairwise Distances

Function Name: `pairwise_distance(A: np.ndarray, B: np.ndarray) -> float`

Description:
You are given two 2D NumPy arrays, `A` and `B`, representing two sets of points in 2D space. Compute the Euclidean distance between every pair of points (i, j) where i is from A and j is from B. Each pair (i, j) is considered only once — symmetric duplicates are not counted again. Return the **average of all unique pairwise distances**.

Mathematical Expression:

$$d(i, j) = \sqrt{(A_i[0] - B_j[0])^2 + (A_i[1] - B_j[1])^2}$$

Then compute the mean over all unique (i, j) pairs.

Input:

- `A`: NumPy array of shape (M, 2)
- `B`: NumPy array of shape (N, 2)

Output:

- A single float — the average of all unique pairwise Euclidean distances

Example:

```
A = np.array([[0, 0], [1, 1]])
B = np.array([[2, 2], [3, 3]])
# Distances: sqrt((0-2)^2+(0-2)^2), sqrt((0-3)^2+(0-3)^2),
#            sqrt((1-2)^2+(1-2)^2), sqrt((1-3)^2+(1-3)^2)
# => [2.828, 4.243, 1.414, 2.828]
# Average = 2.828
```

Section B: Pandas Data Puzzles

Use the following DataFrame as an **example** to understand the schema for upcoming questions. The actual DataFrame used during testing will have the same structure.

```
import pandas as pd
import numpy as np

data = {
    'OrderID': [101, 102, 103, 104, 105, 106, 107, 108],
    'OrderDate': ['2025-10-01', '2025-10-01', '2025-10-02',
                  '2025-10-02', '2025-10-03', '2025-10-04',
                  '2025-10-04', '2025-10-05'],
    'Product': ['Laptop', 'Mouse', 'Keyboard', 'Laptop', 'Monitor',
                'Mouse', 'Webcam', 'Keyboard'],
    'Category': ['Electronics', 'Peripherals', 'Peripherals',
                 'Electronics', 'Electronics', 'Peripherals',
                 'Peripherals', 'Peripherals'],
    'Price': [1200, 25, 75, 1250, 300, 30, 50, 80],
    'Quantity': [2, 5, 3, 1, 2, 4, 1, 2]
}
df = pd.DataFrame(data)
df['OrderDate'] = pd.to_datetime(df['OrderDate'])
```

—

Question 4: The Daily Top Product

Function Name: `get_daily_top_product(df: pd.DataFrame) -> list`

Description:
For each unique `OrderDate`, find the product that generated the highest total revenue (`Price` ×

`Quantity`). Return a list of product names corresponding to each unique date, ordered by ascending `OrderDate`.

Input:

- `df`: DataFrame containing at least the columns `OrderDate`, `Product`, `Price`, and `Quantity`. The `OrderDate` column should be of datetime dtype (or parseable to datetime).

Output:

- **List[str]**: Product names — one per unique date (dates in ascending order).

Notes / Tie-breaking:
If two products on the same date have exactly the same total revenue, the tie is broken deterministically by choosing the product that appears first after sorting by revenue descending and then by product name ascending.

—

Question 5: The Product Relationship Puzzle

Function Name: `strongest_product_pair(df: pd.DataFrame) -> tuple`

Description:
Find the pair of products that are most frequently bought together, i.e., appear in the same `OrderID`. Each unique order can contain multiple products, and you must identify the two products that co-occur most often across all orders.

Return the names of the two products as a tuple `(product1, product2)`. The pair should be ordered alphabetically, i.e., `product1 < product2`.

Input:

- `df`: Pandas DataFrame with columns `OrderID`, `Product`, `Category`, `Price`, and `Quantity`.

Output:

- **Tuple[str, str]** — the names of the two products most frequently co-purchased.

Notes:

- Each `OrderID` can have one or more products.
- You should only consider pairs of *distinct* products.
- If two pairs have the same frequency, choose the pair that is lexicographically smallest (alphabetical order).

—

Question 6: The Dominant Category Challenge

Function name: `most_frequent_leader(df: pd.DataFrame) -> str`

Use the following DataFrame as an **example** to understand the schema for this question. The actual DataFrame used during testing will have the same structure.

```
import pandas as pd
import numpy as np

data = {
    'OrderID': [101, 102, 103, 104, 105, 106, 107, 108],
    'OrderDate': ['2025-10-01', '2025-10-01', '2025-10-02',
                  '2025-10-02', '2025-10-03', '2025-10-04',
                  '2025-10-04', '2025-10-05'],
    'Product': ['Laptop', 'Mouse', 'Keyboard', 'Laptop', 'Monitor',
                'Mouse', 'Webcam', 'Keyboard'],
    'Category': ['Electronics', 'Peripherals', 'Peripherals',
                 'Electronics', 'Electronics', 'Peripherals',
                 'Peripherals', 'Peripherals'],
    'Price': [1200, 25, 75, 1250, 300, 30, 50, 80],
    'Quantity': [2, 5, 3, 1, 2, 4, 1, 2]
}
df = pd.DataFrame(data)
df['OrderDate'] = pd.to_datetime(df['OrderDate'])
```

—

For each date, compute cumulative revenue for each category up to that date. Then determine which category leads (i.e., has the highest cumulative revenue) most frequently across all dates. Return the category name as a string.

Input:

- `df`: Pandas DataFrame with columns `OrderDate`, `Category`, `Price`, and `Quantity`.

Output: String — most frequent leader in cumulative revenue.

—

Section C: Integrated NumPy and Pandas Problems

Question 7: The Sales Simulation

Function name: `simulate_sales(sales_df: pd.DataFrame, days: int = 30) -> list`

Simulate sales data for the next `days` (default = 30).

For each day, generate:

- A price = last known price + random noise from Normal(0, 5)
- A quantity = random integer between 1 and 5

Return a **list of tuples**, where each tuple represents one simulated sale: `(price, quantity)` for each simulated day.

Input:

- `sales_df`: Pandas DataFrame with columns `OrderDate`, `Product`, `Price`.
- `days`: integer (default 30) — number of simulated days.

Output: List of tuples: `[(price1, qty1), (price2, qty2), ...]` of length `days`.

—

Question 8: The Weighted Moving Average (WMA)

Function name: `weighted_moving_average(df: pd.DataFrame, weights: np.ndarray) -> list`

Compute the **Weighted Moving Average (WMA)** of total daily revenues per category across all days. The WMA assigns more weight to recent days (e.g., with weights [0.5, 0.3, 0.2], the latest day gets 0.5).

Return a list containing the WMA value for each day. For the first few days (where there are fewer than the required number of previous days), compute the WMA using the available subset of weights (normalize them so they still sum to 1).

Input:

- `df`: Pandas DataFrame with columns `['OrderDate', 'Category', 'Price', 'Quantity']`.
- `weights`: list or NumPy array of weights (most recent day has the first weight).

Output: List of floats — one WMA value per day.

—